CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

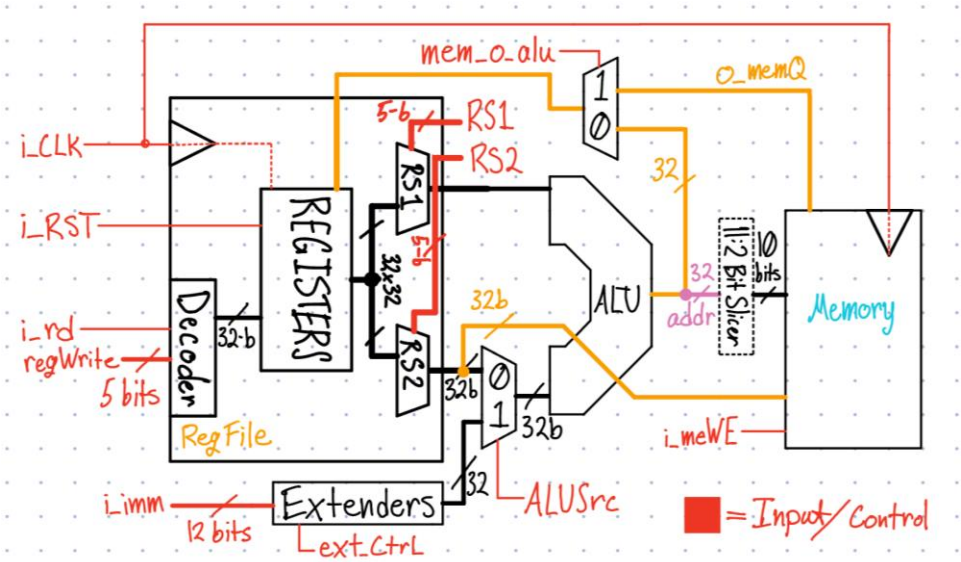
Team Members: __ Jack Dustin _____

___Isaiah Pridie _____

Project Teams Group #:_____ C 07 _____

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.



[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an *N**M* table where each row corresponds to the output of the control logic module for a given instruction.

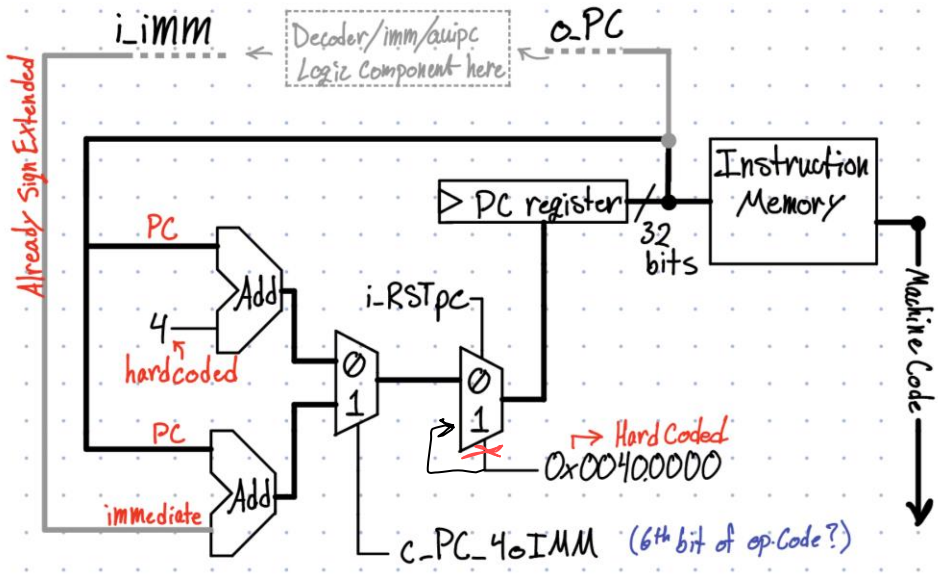
[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).

[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?

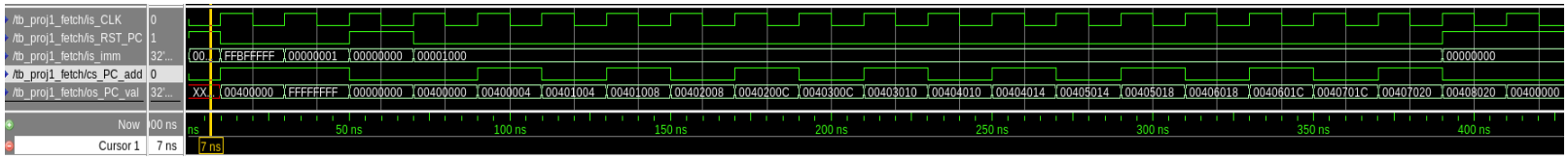
Control Signals Needed:

a. c_PC_4oIMM: Used to control adding by 4 or an immediate (for jumps, etc)
b. i_RSTpc: Used to reset PC register to a specific value (0x0040.0000)
c.



*** Note to us: Insert other Images / Controls if needed ***

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.



What is going on:

1st Cycle (10 ns): Reset PC register
2nd Cycle (30 ns): Use Imm to set PC register to all 1's
3rd Cycle (50 ns): Add 1 to overflow PC register to 0
4th Cycle (70 ns): Reset PC register again

Cycles [5:16] (90 to 390 ns): Use a loop and alternate adding 0x4 (default increment) and adding 0x100 with the immediate.

17th Cycle (410 ns): Reset PC register one last time

*** Note to us: Insert other Waveforms / descriptions if needed ***

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

- Logical shifts will zero extend
- Arithmetic Shifts will sign extend if shifting a signed number

A sla implies an instruction that shifts left, and sign extends. But if you are shifting left then those signs you want to extend are getting overwritten anyways by shifting. The sll instruction already takes care of shifting left. You cannot sign extend a shift left unless you for some reason puts 1's on the LSB side instead of 0's

[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

The difference between arithmetic and logical shifting is sign vs zero extending.

We have a 1-bit signal called s_ex ("signal extender"). This signals value (1 or 0) is the value used to extend back up to 32 bits after a shift.

To choose the value for it we do: "c_extend AND c_Shift AND i_vect_A(31)" Where c_extend is the 5th bit of funct7, c_shift is the 3rd bit of funct3, and i_vect_A(31) is the MSB of the first vector input (input from RS1).

The 3 way and is basically checking:

- "Is this a logical or arithmetic shift?" (c_extend)
- "Am I shifting right or left?" (c_Shift)
- "Is the number being shifted actually negative?" (i_vect_A(31))

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

There are a couple of straight forward ways to implement both right and left shifting:

- Double the hardware so you have 1 barrel shifter for shifting right and 1 for shifting left
- For either left or right shifts (pick one) reverse the order of the input vector, shift like normal, and reverse back before outputting
    - This is the way our barrel shifter is implemented.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?
Wdym "How is Zero *calculated*?" There is a register we can use for 0. This register is x0.
From Duwe's slides: "[slt] produces a 1 if rs < rt and 0 otherwise." slt can be implemented by doing rs – rt and saving the MSB of that calculation.
Cases:

Let rs > rt.	Let rs = 5, rt = 2.	rs – rt = 5 – 2 = 3 = positive. Return 0.
Let rs < rt.	Let rs = 2, rt = 5.	rs – rt = 2 – 5 = -3 = negative. Return 1.
Let rs = rt.	Let rs = 3, rt = 3.	rs – rt = 3 – 3 = 0 = positive. Return 0.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?